

Contents

1	API 設計書 (メイン)	1
1.1	1. 文書概要	1
1.1.1	1.1 目的	1
1.1.2	1.2 対象範囲	1
1.1.3	1.3 版数	1
1.1.4	1.4 関連ドキュメント	2
1.2	2. API 一覧	2
1.2.1	2.1 機能と API の対応 (§ 6.3)	2
1.2.2	2.2 外部 I/F 一覧 (§ 8.1)	2
1.3	3. 共通仕様	3
1.3.1	3.1 API 共通仕様 (§ 8.2)	3
1.3.2	3.2 オーナー境界判定	3
1.3.3	3.2a 論理削除フィルタ	3
1.3.4	3.3 幂等性キー	3
1.3.5	3.4 Webhook 共通仕様 (§ 8.7.3)	3
1.3.6	3.5 連携 IF エンドポイント命名規約 (§ 8.8)	3
1.3.7	3.6 認証ヘッダ	4
1.4	4. レート制限・キャパシティ (§ 11.7 リミット設計表の正本)	4
1.5	5. API 詳細	5
1.5.1	5.1 認証 API	5
1.5.2	5.2 利用者(メンバー)API	7
1.5.3	5.3 プロジェクト管理 API	9
1.5.4	5.4 FAQ 管理 API	11
1.5.5	5.5 ウィジェット API	11
1.5.6	5.6 未解決質問 API	12
1.5.7	5.7 チャット API	12
1.5.8	5.8 利用量・課金 API	13
1.5.9	5.9 お知らせ受信箱 API	13
1.5.10	5.10 規約・退会 API	14
1.5.11	5.11 AI 推論 IF(AnswerProvider 抽象化)	14
1.5.12	5.12 メール配信 IF(EmailProvider 抽象化)	14
1.5.13	5.13 連携 IF(受信側、メイン側エンドポイント)	14
1.5.14	5.14 外部 Webhook(Resend 等)	18
1.6	6. 連携 IF 通信仕様一覧	18
1.7	7. 未決事項	18

1 API 設計書 (メイン)

1.1 1. 文書概要

1.1.1 1.1 目的

メインシステムのすべての API(管理 API / ウィジェット API / 連携 IF #1~#12 送信側 / 外部 Webhook) について、エンドポイント / 認証・認可 / リクエスト / レスポンス / ステータスコード / エラー仕様を一元化する。

1.1.2 1.2 対象範囲

- 対象: 利用者向け管理 API(認証 / プロジェクト / FAQ / 案件 / チャット / ウィジェット設定 / 利用量・課金 / ユーザー管理 / お知らせ / 規約)、エンドユーザー向けウィジェット API、AI 推論抽象化 IF、メール配信抽象化 IF、顧客管理システムとの連携 IF #1~#12(送信側)、外部 Webhook(Resend 等)
- 対象外: 運営者管理 API([02_運営者システム/個別設計書群/03_API 設計書.md](#))、Stripe Webhook 一次受信(運営者側 § 13)

1.1.3 1.3 版数

項目	値
版数	1.0
更新日	2026-05-17

1.1.4 1.4 関連ドキュメント

ドキュメント名	役割	参照先
索引	11 ドキュメント体系の俯瞰	00_ 索引.md
画面設計書	API を呼び出す画面	02_ 画面設計書.md
テーブル定義書	API が操作するテーブル	04_ テーブル定義書.md
認証・認可設計書	認証要否 / 認可判定ルール	09_ 認証認可設計書.md
エラー設計書	ステータスコード / エラー詳細	06_ エラー設計書.md
メッセージ一覧	エラー文言	07_ メッセージ一覧.md
課金・請求設計書	課金 API / IF #1 / IF #10	11_ 課金請求設計書.md
運営者側 API 設計書	連携 IF #1～#12 受信側	../02_ 運営者システム/個別設計書群/03_API 設計書.md

1.2 2. API 一覧

1.2.1 2.1 機能と API の対応 (§6.3)

機能	API エンドポイント	認可	関連画面
認証	/auth/register, /auth/login, /auth/logout, /auth/re-auth, /auth/password/reset-requests	公開 (login)、認証済み (他)	SCR-001, SCR-002, SCR-003
メール確認	/auth/email-verifications/{token}	トークン認証	SCR-023
管理者ユーザー管理	/members/{id}/resend-invitation(招待再送), /projects/{id}/members(SCR-017-M1 招待), /projects/{id}/members/{userId}(ロール変更 / アカウント全体論理削除)	オーナー / プロジェクト管理者 (該当 PJ 範囲)	SCR-017, SCR-017-M1
プロジェクト管理	/projects, /projects/{id}	オーナー専有	SCR-010, SCR-010-M1
FAQ 管理	/projects/{id}/faqs, /projects/{id}/faqs/{faqId}, /projects/{id}/faqs/import, /projects/{id}/faqs/export	オーナー / 該当 PJ の member+	SCR-012
ウィジェット	/widget/bootstrap, /widget/ask, /widget/feed-back	end_user(公開キー + セッション)	ウィジェット
未解決質問	/inquiries, /inquiries/{id}	オーナー / 該当 PJ の member+	SCR-011
チャット	/chat/rooms, /chat/rooms/{id}, /chat/rooms/{id}/messages	admin / end_user	SCR-013
通知	/webhooks/resend	署名検証のみ	(Resend Webhook)
利用量・課金	/usage, /billing/summary, /billing/subscription, /billing/invoices, /billing/budget	ダッシュボード閲覧: オーナー / 該当 PJ の member+, 課金操作: オーナー専有	SCR-015
データ管理	/data/export	admin / オーナー専有	SCR-024
お知らせ受信箱	/me/announcements, /me/announcements/{id}, /me/announcements/{id}/read	admin 限定	SCR-021, SCR-022
規約	/terms/current, /terms/agree	admin	SCR-018, SCR-025
再入室	/widget/sessions/:token	トークン認証	SCR-027

1.2.2 2.2 外部 I/F 一覧 (§8.1)

カテゴリ	接続先	プロトコル	認証方式
顧客管理システム	運営者側システム	HTTPS + mTLS + JWT	連携 IF #1～#12
Stripe	課金決済	HTTPS	API キー、Webhook 署名

カテゴリ	接続先	プロトコル	認証方式
Resend	メール配信	HTTPS	API キー、Webhook 署名
Cloudflare Workers AI	AI 推論	HTTPS(内部)	自動認証
ウィジェット埋込元サイト	顧客サイト	HTTPS + CORS	公開キー + セッショントークン

1.3 3. 共通仕様

1.3.1 3.1 API 共通仕様 (§8.2)

項目	値
Base URL	/api/v1(管理)/ /widget/v1(ウィジェット)/ /internal/admin-integration/v1(内部連携)
データ形式	JSON
エラー形式	RFC 7807(application/problem+json)
日時	ISO 8601 + 末尾 Z(UTC)
ID 形式	ULID(26 文字)、Stripe ID 例外
ページング	カーソル方式(cursor, limit 50~200)
CSRF	Double Submit Cookie 検証(状態変更 API)
Cookie	Secure; HttpOnly; SameSite=Lax; Path= /

1.3.2 3.2 オーナー境界判定

すべての操作で `actor.contract_owner_user_id == target.contract_owner_user_id` を必須(認証・認可設計書 09 § 5.2)。違反時は **404 偽装**(エラー設計書 06 § 5.3)。

1.3.3 3.2a 論理削除フィルタ

`valid` カラムを持つテーブル(`users / contract_owners / project_users / projects / allowed_domains / faqs / question_logs / inquiries / end_users / chat_rooms / billing_subscriptions`)に対する全 GET 系 API(一覧 / 詳細)のクエリは、原則として `WHERE <table>.valid=1` フィルタを追加する。論理削除済み(`valid=0`)の行は通常 API 経由では返却しない。

例外:

- ・ 運営者経由の連携 IF #4(物理削除前の復元 / リストア)では `valid=0` 行への参照を許可
- ・ 監査ログ参照系 API は `valid` 概念外(`audit_logs` には `valid` カラムを追加しない)
- ・ 認証ミドルウェアは `users.valid=0` 行へのログインを 401 で拒否する(セッション内 `userId` の `valid` を確認、`valid=0` ならセッションも失効扱い)

`WHERE valid=1` フィルタの漏れは IDOR 類似の脆弱性として扱う(09_セキュリティ設計.md)。

1.3.4 3.3 冪等性キー

書き込み系 API は **Idempotency-Key** ヘッダ(ULID 推奨、24h 保管)を受け付ける。重複時は既存処理結果を 200 で返却(リトライ安全性)。

1.3.5 3.4 Webhook 共通仕様 (§8.7.3)

項目	仕様
署名検証	HMAC-SHA256(送信元との共有 Secret)
冪等性	event_id で重複処理防止
リトライ	3 回(指数 BO: 1m → 4m → 16m)、超過で DLQ
DLQ 保持	4 日(Cloudflare Queues)、その後 R2 退避 30 日
タイムアウト	30 秒(超過で 5xx 返却)

1.3.6 3.5 連携 IF エンドポイント命名規約 (§8.8)

- ・ 受信: `/internal/admin-integration/v1/{resource}/{action}`
- ・ 送信: `https://<admin-host>/internal/main-integration/v1/...`

- ・ リソース: snake_case 単数形
- ・ 動詞: POST(状態変更)

1.3.7 3.6 認証ヘッダ

ヘッダ	用途	例
Cookie: session=<token>	利用者セッション	管理 API
Authorization: Bearer <wst_*>	ウィジェットセッション	ウィジェット API
Authorization: Bearer <JWT>	連携 IF	内部連携
X-CSRF-Token: <token>	CSRF 対策	状態変更系
X-Chat-Token: <token>	個別チャット再入室	/widget/sessions/:token
Idempotency-Key: <ULID>	冪等性	書き込み系

1.4 4. レート制限・キャパシティ (§11.7 リミット設計表の正本)

項目	警告レベル	拒否レベル	契約上書き	超過時の挙動	正本
同時アクティブ契約	150 でシャーディング着手	200	不可	新規契約受付一時停止 + 運営者通知	NFR-110
FAQ 件数 (契約共通)	8,000(80%)	12,000(120%)	不可	拒否レベルで 429	FR-046
FAQ 質問文字数	-	500 文字	不可	超過は 400	FR-046
FAQ 回答文字数	-	5,000 文字	不可	超過は 400	FR-046
月間質問数	80%(通知)	100%(事後課金、拒否しない)	契約別上書き可	125% で追加制限	FR-122
Workers AI 月間コスト	80%	100%(停止)	契約別上書き可	同上	FR-122
チャット投稿 (EU)	-	10 件/分、2,000 字/msg	不可	429、UI 警告	FR-090
チャット投稿 (admin)	-	無制限 (運営者調整可)	可 (IF #5)	429	FR-090
公開 API /widget/ask	-	60 req/min/IP	可 (IF #5)	429	FR-128
公開 API /widget/bootstrap	-	30 req/min/IP	可 (IF #5)	429	FR-128
ログイン失敗	-	5 回/15 分 (IP × user_id ロック)	不可	アカウントロック	FR-007
管理者ユーザー数	80%	100%	可 (プラン依存)	429、通知	FR-021
プロジェクト数	平均 ×3 超過時通知	-	-	急増検知通知のみ	FR-035
同時オンライン管理者	-	1,000 名	-	503 + Retry-After	NFR-114
ウィジェット同時接続 (EU)	-	10,000 並列	-	503 + Retry-After	NFR-114
D1 容量	8GB(80%)	10GB	-	高優先度アラート + シャーディング検討	NFR-115
Workers CPU 時間	50ms 目標 / 100ms 警告	-	-	100ms 超過が 1h で 1% 超で通知	NFR-116

詳細は 11_課金請求設計書.md § 8 80/100/125% 三段階アクション参照。

1.5 5. API 詳細

1.5.1 5.1 認証 API

1.5.1.1 5.1.1 POST /auth/signup(新規登録)

項目	内容
機能 ID	F-001
認証要否	不要
関連画面	SCR-002
関連テーブル	accounts, terms_agreements

リクエスト:

```
{
  "email": "admin@example.com",
  "password": "P@ssw0rd123!",
  "passwordConfirm": "P@ssw0rd123!",
  "agreedTerms": true,
  "agreedPrivacy": true,
  "turnstileToken": "..."
}
```

レスポンス (202):

```
{ "ok": true }
```

エラー: 400 **VALIDATION_ERROR**(E-INPUT-*), 400 **TURNSTILE_FAILED**

1.5.1.2 5.1.2 POST /auth/login

項目	内容
機能 ID	F-002
認証要否	不要
関連画面	SCR-001
関連テーブル	accounts, sessions

リクエスト:

```
{
  "email": "admin@example.com",
  "password": "...",
  "turnstileToken": "..."
}
```

レスポンス (200):

```
{
  "userId": "01HZ...",
  "role": "admin",
  "requireTermsAgreement": false,
  "activeSessions": [
    { "id": "...", "ipAddress": "203.0.113.x", "createdAt": "..." }
  ]
}
```

エラー: 401 **INVALID_CREDENTIALS**(E-AUTH-CREDENTIAL), 423 **LOCKED_OUT**(E-AUTH-LOCKED), 400 **TURNSTILE_REQUIRED**, 403 **CONTRACT_SUSPENDED**(E-BILL-CONTRACT-SUSPENDED)

1.5.1.3 5.1.3 POST /auth/logout | 認証 | Cookie + CSRF | | レスポンス | 200 { "ok": true } |

1.5.1.4 5.1.4 POST /auth/password-reset-request(パスワード再設定要求) | 機能 ID | F-004 | | 認証
要否 | 不要 | | 関連画面 | SCR-003 |

リクエスト: { "email": "admin@example.com", "turnstileToken": "..." } レスポンス (202): { "ok": true } (列挙攻撃対策で存在有無を返さない)

1.5.1.5 5.1.5 POST /auth/re-auth(再認証) | 機能 ID | F-005 | | 認証要否 | 要 |

リクエスト: { "password": "..." } レスポンス (200): { "ok": true, "token": "reauth_..." } エラー: 401 INVALID_PASSWORD

1.5.1.6 5.1.6 POST /auth/email-verifications/{token}(メール確認) | 機能 ID | F-003 | | 関連画面 | SCR-023 |

トークン形式 = email_verify purpose の access_tokens.token_hash 検証。レスポンス (200): { "userId": "...", "verifiedAt": "..." } エラー: 410 TOKEN_EXPIRED(E-AUTH-TOKEN-EXPIRED)

1.5.1.7 5.1.7 POST /auth/invitations/{token}/preview(招待トークン検証 + 招待情報プレビュー) | 機能 ID | F-016 | | 認証 | 未認証可 (トークン認証のみ) | | 関連画面 | SCR-031 メンバーアカウント有効化 (初期表示時) |

トークン形式 = activation purpose の access_tokens.token_hash 検証。検証成功時は招待情報 (プロジェクト名 / 付与ロール / 招待元オーナー名 / 招待者メールアドレス) を返す。SCR-031 の招待情報パネル表示に使用する。Turnstile 不要 (プレビューは情報取得のみで状態変更なし)。

レスポンス (200):

```
{
  "userId": "01HZ...",
  "email": "newmember@...",
  "projectId": "proj_1",
  "projectName": " 顧客サポート",
  "invitedRole": "admin" | "member",
  "inviterOwnerName": " 山田 一郎",
  "expiresAt": "2026-05-28T..."
}
```

エラー: 410 TOKEN_EXPIRED(E-AUTH-TOKEN-EXPIRED)、410 TOKEN_USED(E-AUTH-TOKEN-USED)、404 TOKEN_NOT_FOUND

1.5.1.8 5.1.8 POST /auth/invitations/{token}/activate(メンバーアカウント有効化) | 機能 ID | F-016 | | 認証 | 未認証可 (トークン認証のみ、Turnstile 必須) | | 関連画面 | SCR-031 メンバーアカウント有効化 (「登録を完了する」押下時) |

招待されたメンバー本人が氏名 (displayName) + 初回パスワード (password) + 利用規約同意 (termsAgreed) + プライバシーポリシー同意 (privacyAgreed) を受け取り、同一トランザクションで以下を実行する:

1. verifyToken(rawToken, 'activation', env) → 期限・使用済みチェック
2. access_tokens.meta から invitedProjectId / invitedRole を取得
3. UPDATE users SET name=?, password_hash=?, status='active', updated_at=now() WHERE id=?
4. INSERT INTO terms_agreements (user_id, terms_version, privacy_version, agreed_at) VALUES (...)
5. UPDATE project_users SET valid=1, updated_at=now() WHERE user_id=? AND project_id=?
6. consumeToken(rawToken, 'activation', env) → access_tokens.used_at セット
7. writeAudit('user.activation_completed', { user_id, project_id, role })

リクエスト:

```
{
  "displayName": " 山田 太郎",
  "password": "P@ssw0rd-strong",
  "passwordConfirm": "P@ssw0rd-strong",
  "termsAgreed": true,
  "privacyAgreed": true,
}
```

```
"turnstileToken": "..."  
}
```

バリデーション:

- **displayName**: 必須、1~100 文字 (前後空白トリム)
- **password**: 12 文字以上、英大文字 / 小文字 / 数字 / 記号のうち 3 種類以上 (FR-006)
- **passwordConfirm**: **password** と一致
- **termsAgreed** / **privacyAgreed**: 両方 **true** 必須 (FR-160 / FR-164)
- **turnstileToken**: 検証成功 (FR-177)

レスポンス (200): { "userId": "01HZ...", "redirectUrl": "/auth/login" } エラー: 410 **TOKEN_EXPIRED**(E-AUTH-TOKEN-EXPIRED)、410 **TOKEN_USED**(E-AUTH-TOKEN-USED)、400 **VALIDATION_ERROR**(氏名長 / パスワード強度 / 規約同意未チェック)、400 **TURNSTILE_FAILED**

1.5.1.9 5.1.9 POST /auth/contact-verifications/{token}(プロジェクト連絡先メール確認) | 機能 ID | F-PRJ-001(プロジェクト管理派生)| | 認証 | 未認証可 (トークン認証のみ、Turnstile 不要)| | 関連画面 | SCR-032 プロジェクト連絡先メール確認完了 |

TPL-PROJECT_CONTACT_VERIFY 確認メール内リンクの着地時に呼び出され、トークン検証成功時に **projects.contact_email_verified_at** をセットする。Turnstile は不要 (入力フォームを持たない単純な確認専用エンドポイントで、トークン HMAC-SHA256 検証 + IP/トークン単位のレート制限で十分)。

同一トランザクションで以下:

1. **verifyToken**(rawToken, 'contact_verify', env) → 期限・使用済みチェック
2. **access_tokens.meta.projectId** を取得 (JSON パース)
3. **UPDATE projects SET contact_email_verified_at=now(), updated_at=now() WHERE id=?**
4. **consumeToken**(rawToken, 'contact_verify', env) → **access_tokens.used_at** セット
5. **writeAudit**('project.contact_email_verified', { project_id, email_hmac })

リクエスト: ボディなし (URL のトークンのみ)

レスポンス (200): { "projectId": "proj_1", "projectName": " サポートサイト", "verifiedAt": "2026-05-21T..." } エラー: 410 **TOKEN_EXPIRED**(E-AUTH-TOKEN-EXPIRED)、410 **TOKEN_USED**(E-AUTH-TOKEN-USED)、404 **TOKEN_NOT_FOUND**

1.5.2 5.2 利用者 (メンバー)API

1.5.2.1 5.2.1 GET /members | 機能 ID | F-015 | | 必要権限 | オーナー / プロジェクト管理者 (該当 PJ 範囲のメンバーのみ可視)| | 関連画面 | SCR-017 |

レスポンス (200):

```
{  
  "items": [  
    {  
      "id": "01HZ...",  
      "email": "member@...",  
      "status": "active|pending_activation",  
      "role": "admin",  
      "isOwner": false,  
      "projectGrants": [  
        { "projectId": "proj_1", "role": "admin" },  
        { "projectId": "proj_2", "role": "member" }  
      ],  
      "lastLoginAt": "2026-05-13T10:00:00Z",  
      "invitationExpiresAt": "2026-05-20T..."  
    }  
  ],  
  "nextCursor": "01HZ..." | null  
}
```

非オーナーが呼び出した場合、**items[]** は「操作者自身が **admin** を持つプロジェクトに割当のあるメンバー」に絞られ、各 **projectGrants[]** も操作者の **admin** 範囲のプロジェクトのみが見える。

1.5.2.2 5.2.4 POST /projects/{id}/members(プロジェクト単発招待 / SCR-017-M1 招待モード正本) | 認証 | Cookie + CSRF + 再認証必須 | | 必要権限 | オーナー / 該当プロジェクトの admin ロール保持メンバー | | 関連画面 | SCR-017-M1(プロジェクト管理者の招待操作)|

該当プロジェクト 1 件に対する単発招待。既存メンバーアカウントの場合は `project_users` 行のみ追加 (指定ロール)。新規メールアドレスの場合は `users` を `name=NULL / status='pending_activation'` で作成 + `project_users` を `valid=0` で同時 INSERT + 招待メール送信 (7 日有効、`access_tokens.purpose='activation'`、`meta={"invitedProjectId": "...", "invitedRole": "..."}` を JSON 保持)。氏名 (`displayName`) は受け付けない (招待された本人が SCR-031 メンバーアカウント有効化ページで入力する。FR-016d / FR-016e)。

リクエスト:

```
{
  "email": "newmember@...",
  "role": "admin"
}
```

レスポンス (201): { "userId": "01HZ...", "status": "active|pending_activation", "expiresAt": "... | null } エラー: 409 ALREADY_EXISTS_IN_PROJECT(既に該当プロジェクトに割当あり)、400 VALIDATION_ERROR、403 PROJECT_ACCESS_DENIED

1.5.2.3 5.2.4a PATCH /projects/{id}/members/{userId}(プロジェクト単位のロール変更 / SCR-017-M1 編集モード正本) | 認証 | Cookie + CSRF + 再認証必須 | | 必要権限 | オーナー / 該当プロジェクトの admin ロール保持メンバー | | 関連画面 | SCR-017-M1(編集モードの「変更を保存」)|

当該プロジェクトの `project_users.role` を `admin/member` で上書きする。他プロジェクトの割当には影響しない。

リクエスト:

```
{
  "role": "admin" | "member"
}
```

- ・同時に表示名 (`displayName`) を更新する場合は別途 `PATCH /members/{id}` を呼び出す (プロジェクト範囲外の属性のため)
- ・「該当プロジェクトの最後の `admin` を `member` へ降格」する結果になる場合は 403 LAST_ADMIN_PROTECTED(E-AUTHZ-LAST-ADMIN-PROTECTED) で拒否
- ・自分自身が当該プロジェクトの最後の `admin` で、自身を `member` へ降格する場合は 403 SELF_MUTATION_FORBIDDEN
- ・ロール変更後も当該プロジェクトに `project_users` 行が維持される限り、`normal / low` 通知の配信対象に含まれる (配信先はトグル ON 時にプロジェクトの全メンバー)

レスポンス (200): { "userId": "...", "projectId": "...", "role": "admin|member", "updatedAt": "..." } エラー: 404 NOT_FOUND、403 PROJECT_ACCESS_DENIED、403 LAST_ADMIN_PROTECTED、403 SELF_MUTATION_FORBIDDEN、400 VALIDATION_ERROR

1.5.2.4 5.2.5 DELETE /projects/{id}/members/{userId}(メンバーアカウント論理削除) | 認証 | Cookie + CSRF + 再認証必須 | | 必要権限 | オーナー / 該当プロジェクトの admin ロール保持メンバー | | 関連画面 | SCR-017-M1「アカウントを削除」(L2) |

アカウント全体論理削除を行う。同一トランザクションで次を実行する:

1. `UPDATE users SET valid=0, updated_at=now() WHERE id=?`(対象 `users` 行を論理削除)
2. `UPDATE project_users SET valid=0, updated_at=now() WHERE user_id=? AND valid=1`(対象アカウントの全 `project_users` 行を論理削除、当該プロジェクトに限らず他プロジェクト分も含む)
3. `UPDATE sessions SET revoked_at=now() WHERE user_id=? AND revoked_at IS NULL`(全セッション失効)
4. 招待中の場合は対応する `access_tokens.purpose='activation'` を `used_at=now()` で強制失効

API パスはプロジェクトスコープ表記を維持 (操作起点が「当該プロジェクトの SCR-017-M1」であるため認可境界判定に `projectId` が必要)。`userId` は実際にはアカウント全体の論理削除を意味する。論理削除データは `updated_at` 基準で 90 日経過後に物理削除バッチ (DD14) で物理削除される (物理削除モードに従って実体を消去)。

同一メールアドレスでの再招待: 契約内メール一意性はアプリ層トランザクションで担保 (03_テーブル設計.md § 3.1 を正本)。論理削除済 (**valid=0**) の **users** 行を維持したまま、同一オーナー配下の同一メールで新規 **users** 行を **valid=1** で作成可能。

権限ルール (FR-019 / FR-019a / FR-019b):

操作者	削除可	削除不可
オーナー (contract_owners 行存在)	当該プロジェクトに割当のある admin / member メンバー (他プロジェクトにも割当があれば連鎖論理削除)	オーナー自身 (自分)
プロジェクト管理者 (contract_owners 行を持たない + 当該プロジェクトに admin)	当該プロジェクトに割当のある member メンバー	自分自身 / 当該プロジェクトの他の admin (E-AUTHZ-ADMIN-DELETE-PROTECTED)

最後の admin の有無に関する振る舞い (FR-019a): オーナーが作成時に自動付与される **project_users(role='admin', valid=1)** 行を常に保持するため、「プロジェクトに admin 行が 0 件になる」状態は構造的に発生しない。E-AUTHZ-LAST-ADMIN-PROTECTED の判定はオーナー由来 admin 行除外 (**pu.user_id NOT IN (SELECT user_id FROM contract_owners WHERE valid=1) AND role='admin' AND valid=1** の件数) で行う。

レスポンス (204): No Content エラー: 403 E-AUTHZ-OWNER-PROTECTED(対象がオーナー)、403 E-AUTHZ-SELF-MUTATION(自分自身対象)、403 E-AUTHZ-ADMIN-DELETE-PROTECTED(オーナー以外が当該プロジェクトの他 admin を削除しようとした)、403 PROJECT_ACCESS_DENIED、404 NOT_FOUND、409 E-BIZ-ACCOUNT-INACTIVE(対象が既に **valid=0**)

1.5.2.5 5.2.6 POST /members/{id}/resend-invitation(招待メール再送) | 認証 | Cookie + CSRF + 再認証必須 | | 必要権限 | オーナー / 該当プロジェクトの admin 保持メンバー | | 関連画面 | SCR-017-M1(招待再送)|

status='pending_activation' のメンバーのみ対象。旧 **access_tokens.purpose='activation'** を失効させ新規トークン (7 日有効、**meta** を同一の **invitedProjectId/invitedRole** で再発行) を発行。リクエストボディは空 (氏名 **displayName** を受け付けない。FR-016b / FR-016d)。

エラー: 403 E-AUTHZ-OWNER-PROTECTED、403 E-AUTHZ-SELF-MUTATION、404 NOT_FOUND

1.5.3 5.3 プロジェクト管理 API

1.5.3.1 5.3.1 GET /projects | 機能 ID | F-034 | | 関連画面 | SCR-010 | | 必要権限 | オーナー専有 |

レスポンス (200):

```
{
  "items": [
    {
      "id": "01HZ...",
      "name": "...",
      "status": "active|deleted",
      "createdAt": "...",
      "faqCount": 42,
      "widgetKey": "pk_live...",
      "allowedDomains": ["example.com", "*.example.com"],
      "contactEmail": "contact@...",
      "contactEmailVerifiedAt": "..." | null
    }
  ],
  "nextCursor": null
}
```

1.5.3.2 5.3.2 POST /projects(新規作成) | 認証 | Cookie + CSRF | | 必要権限 | オーナー専有 (プロジェクト設定全般を含めオーナーのみ実行可) | | 関連画面 | SCR-010-M1(新規作成モード)|

リクエスト:

```
{
  "name": " 新規プロジェクト",
  "description": " 説明文",
  "allowedDomains": ["example.com"]
}
```

オーナー固定化 (FR-030a / FR-015e):

- ・ リクエストに `initialAdmins` フィールド (`selfAsAdmin` フラグや `adminEmails[]`) は持たない。オーナーは作成時に自動で当該プロジェクトの管理者となるため、UI / API での個別指定は不要。
- ・ サーバー側処理: プロジェクト作成成功と同一トランザクション内で `INSERT INTO project_users(user_id=actor.userId, project_id=newPid, contract_owner_user_id=actor.contractOwnerUserId, role='admin', valid=1, granted_at=now(), granted_by=actor.userId)` を 1 行発行する (オーナー自動 admin 行付与)。認可上は `isOwner=true` bypass を維持しつつ、本 admin 行は critical 通知の宛先解決 (`SELECT DISTINCT user_id FROM project_users WHERE role='admin' AND valid=1 ...`) と画面表示の事実情報として参照される。
- ・ 他者をプロジェクト管理者として招待する操作は、プロジェクト作成後に当該プロジェクトの SCR-017 / SCR-017-M1(プロジェクト WS) から `POST /projects/{id}/members` 経由で行う。

レスポンス (201):

```
{
  "id": "01HZ...",
  "widgetKey": "pk_live_abc123...",
  "keyExpiresAt": "2027-05-13T..."
}
```

エラー: 400 `VALIDATION_ERROR`(name / allowedDomains 必須違反)、409 `DUPLICATE_NAME`、403 `E-AUTHZ-OWNER-ONLY`

1.5.3.3 5.3.3 PATCH /projects/{id} / DELETE /projects/{id} | 認証 | Cookie + CSRF(削除は再認証必須、L3 = プロジェクト名タイプ確認 + パスワード再認証) | 必要権限 | **オーナー専有** | 関連画面 | **PATCH: SCR-010-M1 編集モード / DELETE: SCR-026 のみ** | エラー | 404 `NOT_FOUND`(オーナー境界違反偽装)、403 `E-AUTHZ-OWNER-ONLY` |

リクエスト (PATCH、部分更新):

```
{
  "name": "...",
  "allowedDomains": [...],
  "contactEmail": "..."
}
```

DELETE /projects/{id} 実行時の挙動 (FR-030b、論理削除):

1. `UPDATE project_users SET valid=0, updated_at=now() WHERE project_id=? AND valid=1`(オーナー自身の admin 行も含む全行を論理削除)
2. `UPDATE` 後、他プロジェクト割当 (`valid=1` 行) が 0 件かつ `contract_owners` 行を持たないメンバーの `users` を `UPDATE users SET valid=0, updated_at=now()` で論理削除 + 全セッション失効 (`UPDATE sessions SET revoked_at=now() WHERE user_id IN (...)`) + 未使用招待トークン失効 (`UPDATE access_tokens SET used_at=now() WHERE user_id IN (...) AND used_at IS NULL`)
3. `UPDATE projects SET status='deleted', valid=0, deleted_at=now(), updated_at=now() WHERE id=?`
4. 関連テーブル (`allowed_domains` / `faqs` / `inquiries` / `end_users` / `chat_rooms` / `question_logs`) の対象行を `valid=0, updated_at=now()` に伝播。匿名化モード (`contract_owners.data_deletion_mode='anonymize'`) の場合の即時匿名化は本処理では行わず、90 日後の物理削除バッチ時に実施する
5. 監査ログ `project.logical_delete` を `retention_class='general'` で記録 (metadata: {`projectId`, `logicallyDeletedUserIds`: [...]})

上記 1-5 はアプリ層のトランザクション内で実装する。論理削除データは `updated_at` 基準で 90 日経過後に物理削除バッチ (DD14) で物理削除される。物理削除時は `ON DELETE CASCADE` で関連行が連鎖物理削除される。

1.5.3.4 5.3.4 POST /projects/{id}/widget-key/rotate(ウィジェット鍵ローテーション) | 認証 | 再認証必須 | 関連画面 | SCR-014 |

リクエスト: { "expiresIn": 7 | 30 | 90 | 180 | 365 } レスポンス (201): { "newKey": "pk_live_xyz789...", "expiresAt": "...", "deprecationWarning": "..."} }

1.5.3.5 5.3.5 GET /projects/{id}/notification-settings(v2.0 で廃止)/ 5.3.6 PATCH /projects/{id}/notification-settings(v2.0 で廃止) v2.0 で SCR-030 を廃止し、プロジェクト関連通知を常時 ON 固定にしたため、本 API 2 種を廃止。プロジェクト情報の参照は SCR-015 プロジェクトホームの「プロジェクト情報」パネルに統合し、編集はオーナー専用の PATCH /projects/{id}(§ 5.3.3、SCR-010-M1) に集約。

1.5.4 5.4 FAQ 管理 API

1.5.4.1 5.4.1 GET /faqs?status=draft&projectId=...&keyword=...&cursor=... | 必要権限 | 編集系: オーナー / 該当 PJ の member+(admin または member) / 参照: 同左 |

1.5.4.2 5.4.2 POST /faqs / PATCH /faqs/{id} / DELETE /faqs/{id} 楽観ロック: version 必須。CONFLICT 時 409。

1.5.4.3 5.4.3 POST /faqs/{id}/publish | 関連画面 | SCR-012 |

リクエスト: { "version": 6 } レスポンス (200): { "id": "...", "status": "published", "version": 7, "publishedAt": "..."} エラー: 422 INVALID_STATE (FAQ 自動公開禁止、§ 4.2.1 ガード)、409 CONFLICT

1.5.4.4 5.4.4 POST /faqs/import リクエスト: multipart/form-data { file: <CSV> } レスポンス (202): { "jobId": "job_...", "status": "processing"} }

1.5.5 5.5 ウィジェット API

1.5.5.1 5.5.1 POST /widget/v1/bootstrap | 認証 | 不要 (ドメイン検証) |

リクエスト:

```
{
  "publicKey": "pk_live_abc123...",
  "origin": "https://example.com"
}
```

レスポンス (200):

```
{
  "sessionToken": "wst_...",
  "expiresIn": 3600,
  "projectConfig": {
    "headerColor": "#06c",
    "placement": "bottom-right",
    "headerTitle": " お問い合わせ",
    "contactEmail": "contact@..."
  }
}
```

エラー: 401 WIDGET_KEY_INVALID / WIDGET_KEY_EXPIRED、403 DOMAIN_NOT_ALLOWED、403 CONTRACT_SUSPENDED

1.5.5.2 5.5.2 POST /widget/v1/ask(質問送信) | 認証 | Bearer session_token | | 機能 ID | F-049-052 |

リクエスト: { "question": " 返品の手続きは?", "projectId": "proj_..." }

レスポンス (200) - answered:

```
{
  "type": "answered",
  "answer": " 返品は商品到着後 7 日以内に...",
  "confidence": 0.78,
  "referencedFaqs": [
    { "id": "faq_...", "question": " 返品ポリシー", "answer": "..." }
  ]
}
```

```

],
"questionLogId": "qlog_..."
}

```

レスポンス (200) - unanswered:

```

{
  "type": "unanswered",
  "reason": "low_confidence" | "no_faq_match" | "pii_detected",
  "questionLogId": "qlog_..."
}

```

エラー: 429 RATE_LIMITED、403 DOMAIN_NOT_ALLOWED

1.5.5.3 5.5.3 POST /widget/v1/inquiries(未解決質問登録) リクエスト:

```

{
  "email": "user@example.com",
  "displayName": " 太郎",
  "questionId": "qlog_...",
  "idempotencyKey": "<UUIDv4>"
}

```

レスポンス (201): { "inquiryId": "inq_...", "inquiryCode": "INQ-20260513-XYZ123", "status": "open" }

未解決質問登録ではチャット部屋を作成しない。

1.5.5.4 5.5.4 POST /widget/v1/chat-rooms リクエスト:

```

{
  "email": "user@example.com",
  "displayName": " 太郎",
  "questionId": "qlog_...",
  "idempotencyKey": "<UUIDv4>"
}

```

レスポンス (201): { "roomId": "room_...", "chatCode": "CHAT-20260513-XYZ123" }

1.5.5.5 5.5.5 POST /widget/v1/chat-rooms/{id}/messages リクエスト: { "body": " メッセージ本文", "idempotencyKey": "<UUIDv4>" } エラー: 429(10/分制限)、400(2,000 字超)

1.5.6 5.6 未解決質問 API

1.5.6.1 5.6.1 GET /inquiries?status=open&projectId=...&cursor=... status クエリは open / closed のいずれか(または未指定で両方)。

1.5.6.2 5.6.2 GET /inquiries/{id} / PATCH /inquiries/{id} PATCH は担当者および状況の手動切替を更新可能。

```

{
  "assigneeUserId": "...",
  "status": "open" | "closed"
}

```

各フィールドはオプション(指定したものだけ更新)。状況は open ⇄ closed の双方向遷移を許容(再オープン制限なし)。FAQ 下書き保存・FAQ 公開・個別チャットの操作は本フィールドを変更しない(連動ロジックなし)。状態変更時は inquiry_status_history に reason='manual' で記録する。

1.5.7 5.7 チャット API

1.5.7.1 5.7.1 GET /chat-rooms/{id}/messages / POST /chat-rooms/{id}/messages

1.5.7.2 5.7.2 POST /chat-rooms / GET /chat-rooms/{id} チャット部屋は chat_code を発行し、未解決質問 ID を受け付けない。部屋の close / reopen API は提供しない。

1.5.8 5.8 利用量・課金 API

1.5.8.1 5.8.1 GET /usage?period=current_month&viewMode=owner|project&projectId={id} | 機能 ID | F-120 | | 関連画面 | SCR-015 | | 必要権限 | viewMode=owner はオーナー専有。viewMode=project はオーナー/該当プロジェクトの admin または member |

viewMode はヘッダーのダッシュボード切替アイコンと連動する。オーナーは owner/project を切替可能。プロジェクト管理者・メンバーは project 固定とし、projectId はヘッダーで選択中のプロジェクトを指定する。

レスポンス (200):

```
{
  "period": "2026-05-01T00:00:00Z",
  "questions": { "used": 750, "limit": 1000, "percentage": 75, "resetAt": "2026-06-01T..." },
  "faqs": { "used": 85, "limit": 100, "percentage": 85 },
  "chatRooms": { "used": 28, "limit": 30, "percentage": 93 }
}
```

1.5.8.2 5.8.2 GET /billing/invoices?limit=6 レスポンス (200):

```
{
  "items": [
    {
      "id": "inv_...",
      "date": "2026-05-01T...",
      "amount": 5000,
      "currency": "JPY",
      "status": "paid|failed|draft",
      "pdfUrl": "https://..."
    }
  ]
}
```

pdfUrl は R2 署名 URL(有効期限 5 分)。

1.5.8.3 5.8.3 PATCH /billing/monthly-budget-limit | 認証 | 再認証必須 | | 機能 ID | FR-127 |

リクエスト: { "limitJpy": 50000 } レスポンス (200): { "limitJpy": 50000, "updatedAt": "..." }

1.5.9 5.9 お知らせ受信箱 API

1.5.9.1 5.9.1 GET /me/announcements?cursor=... | 必要権限 | admin 限定 | | 関連画面 | SCR-021 |

レスポンス (200):

```
{
  "items": [
    {
      "id": "ann_...",
      "title": "...",
      "category": "billing|announcement|system",
      "priority": "low|normal|high|critical",
      "bodyHtml": "<p> 本文 </p>",
      "readAt": "..." | null,
      "createdAt": "..."
    }
  ],
  "nextCursor": null
}
```

1.5.9.2 5.9.2 POST /me/announcements/{id}/read レスポンス (200): { "id": "...", "readAt": "..." }

1.5.9.3 5.9.3 GET /me/announcements/unread-summary レスポンス (200): { "unreadsCount": 5, "recent": [...] }

1.5.10 5.10 規約・退会 API

1.5.10.1 5.10.1 GET /terms/current | 認証要否 | 不要 | | 関連画面 | SCR-018 |

1.5.10.2 5.10.2 POST /terms/agree | 関連画面 | SCR-025 | | 関連テーブル | terms_agreements |

1.5.10.3 5.10.3 POST /withdrawal-requests(退会申請) | 認証 | オーナー専有、再認証必須 | | 関連画面 | SCR-024 | | 関連テーブル | withdrawal_requests, contract_owners |

エラー: 403 PERMISSION_DENIED(メンバー)

1.5.11 5.11 AI 推論 IF(AnswerProvider 抽象化)

```
export type AnswerResult =  
  | { kind: 'answered'; answer: string; cited_faq_ids: string[]; confidence: number }  
  | { kind: 'unanswerable'; reason_code: 'no_match' | 'low_confidence' | 'contradiction' }  
  | { kind: 'error'; reason_code: 'provider_error' | 'timeout' | 'rate_limited' };
```

```
export interface AnswerProvider {  
  generate(input: {  
    question: string;  
    candidate_faqs: Array<{ id: string; question: string; answer: string }>;  
    policy: { faq_only: true; forbid_new_facts: true; learn: false };  
    locale: 'ja-JP';  
    timeout_ms: 8000;  
  }): Promise<AnswerResult>;  
  
  healthcheck(): Promise<{ ok: boolean; provider: string; model: string }>;  
}
```

MVP は Cloudflare Workers AI 実装 (WorkersAIAnswerProvider)。

1.5.12 5.12 メール配信 IF(EmailProvider 抽象化)

```
export interface EmailProvider {  
  send(input: {  
    to: string; from: string; reply_to?: string; subject: string;  
    html: string; text?: string; headers?: Record<string, string>;  
    idempotency_key: string;  
  }): Promise<{ message_id: string }>;  
  
  verifyWebhook(headers: Headers, body: string): Promise<{  
    valid: boolean;  
    event_type: 'sent' | 'delivered' | 'bounced' | 'complained' | 'failed' | 'opened' | 'clicked';  
    message_id: string;  
    timestamp: string;  
  }>;  
}
```

MVP は Resend 実装 (ResendEmailProvider)。

1.5.13 5.13 連携 IF(受信側、メイン側エンドポイント)

1.5.13.1 5.13.1 IF #1 契約停止イベント受信 (顧管 → ×)

POST /internal/admin-integration/v1/owner/suspend

認証: mTLS + JWT(aud=main、TTL 5min)

リクエスト:

```
{
  "operationId": "01HZ...",
  "contractOwnerUserId": "01HZ...",
  "targetStatus": "suspended",
  "reason": "payment_failed|terms_violation|manual_admin",
  "occurredAt": "2026-05-13T10:00:00Z",
  "operatorAccountId": "..."
}
```

ヘッダ: Idempotency-Key: (operation_id, contract_owner_user_id, target_status)

レスポンス (200): { "ok": true, "newStatus": "suspended", "appliedAt": "..." }

動作:

1. JWT + Idempotency-Key 検証
2. **contract_owners.contract_status** 更新
3. KV キャッシュ無効化 (セッション/KV/レート制限)
4. reason に応じてセッション処理 (**operator_manual / tos_violation** は **5 秒以内** に全セッション無効化)

エラー: 401 JWT_INVALID、409 CONFLICT

1.5.13.2 5.13.2 IF #2 強制ログアウト受信

POST /internal/admin-integration/v1/owner/forced-logout

リクエスト:

```
{
  "operationId": "...",
  "contractOwnerUserId": "...",
  "scope": "all|single_account",
  "userId": "...",
  "reason": "..."
}
```

動作: sessions.revoked_at 設定 + KV キャッシュ強制 invalidate(5 秒以内)

1.5.13.3 5.13.3 IF #4 復元実行受信

POST /internal/admin-integration/v1/owner/restore

認証: mTLS + JWT + 再認証トークン

1.5.13.4 5.13.4 IF #5 レート制限上書き受信

POST /internal/admin-integration/v1/rate-limit/override

リクエスト:

```
{
  "contractOwnerUserId": "...",
  "overrideId": "...",
  "limits": [{ "kind": "ask", "limit": 30, "windowSec": 60, "validUntil": "..." }],
  "reason": "...",
  "operatorAccountId": "..."
}
```

動作:

1. JWT + Idempotency-Key 検証
2. **owner_quota_overrides** テーブル更新
3. KV キー **ratelimit:{contractOwnerUserId}:{kind}** 更新 (即時反映)

1.5.13.5 5.13.5 IF #6 AI しきい値上書き受信

POST /internal/admin-integration/v1/threshold/update

リクエスト:

```
{
  "contractOwnerUserId": "...",
  "version": "2026-05-13T10:00:00Z",
  "projectId": null,
  "confidenceThreshold": 0.65,
  "relevanceThreshold": 0.55,
  "scope": "owner|project",
  "operatorAccountId": "..."
}
```

動作:

1. JWT + Idempotency-Key 検証
2. KV `ai_threshold:{contractOwnerUserId}:{projectId}` 更新 (60s TTL)
3. `ai_threshold_persistent_cache` に永続化
4. IF #12 経由で管理者ユーザーに inbox 配信

1.5.13.6 5.13.6 IF #7 お知らせ生成受信

POST /internal/admin-integration/v1/announcement/inbound

リクエスト:

```
{
  "announcementId": "...",
  "contractOwnerUserId": null,
  "title": "...",
  "bodyHtml": "<p>...</p>",
  "importance": "low|normal|high|critical",
  "audienceContractOwnerUserIds": null,
  "publishedAt": "...",
  "operatorAccountId": "..."
}
```

- `audienceContractOwnerUserIds`:
 - null または空配列 → 全契約配信 (`contract_owners.contract_status='active' AND valid=1` を動的解決)
 - 配列 → 列挙された `contract_owner_user_id` への限定配信 (各要素は ULID 形式)

動作:

1. JWT + Idempotency-Key 検証
2. `service_announcements` レコード作成
3. `audienceContractOwnerUserIds` が配列の場合、各要素に対し `announcement_audiences(announcement_id, contract_owner_user_id)` 行を INSERT (配信先は中間テーブル化されており、`service_announcements` 側に JSON 列は持たない)
4. `announcement_recipients` を宛先範囲に応じて生成 (`announcement_audiences` 行が 0 件なら全契約、1 件以上なら列挙された owner のみ)
5. `announcement-fanout-queue` に投入、`inbox_messages` へ fan-out

1.5.13.7 5.13.7 IF #8 監視メトリクス取得 (送信側、メ → 顧管)

GET <https://<admin-host>/internal/main-integration/v1/metrics?window=5m|1h|24h>

認証: mTLS + JWT(aud=admin)

レスポンス (200):

```
{
  "window": "5m",
  "since": "...",
  "until": "...",

```



```

"owners": [
  {
    "contractOwnerUserId": "...",
    "errorRate5xx": 0.002,
    "aiInferenceP95Ms": 1234,
    "queueDlqCount": 0,
    "emailBounceRate": 0.01,
    "emailComplaintRate": 0.0005,
    "billingWebhookFailures": 0,
    "aiAnswerableRate": 0.85
  }
],
"global": { "d1UsageRatio": 0.65 }
}

```

1.5.13.8 5.13.8 IF #9 不正利用検知通知 (送信側、× → 顧管)

POST https://<admin-host>/internal/main-integration/v1/fraud/detected

リクエスト: { "detectionId": "...", "contractOwnerUserId": "...", "pattern": "...", "details": {...} }

DLQ 上限: 200/30 分

1.5.13.9 5.13.9 IF #10 課金 Webhook 受信 (顧管 → ×)

POST /internal/admin-integration/v1/billing-webhook/forward

リクエスト:

```

{
  "stripeEventId": "evt_...",
  "eventType": "invoice.paid|invoice.payment_failed|customer.subscription.created|...",
  "contractOwnerUserId": "...",
  "payload": { /* Stripe イベント */ },
  "receivedAt": "...",
  "signatureVerifiedAt": "..."
}

```

動作:

1. JWT 検証 + event_id 冪等性
2. Stripe イベント種別に応じてハンドラ分岐:
 - invoice.paid: billing_invoices.status='paid'、suspended → active 復帰
 - invoice.payment_failed: 通知、7 日猶予タイマー開始
 - customer.subscription.created: billing_subscriptions 作成
 - charge.refunded: 訂正請求行記録
3. IF #1 経由で contract_owners.contract_status 反映

Idempotency-Key: event_id(Stripe evt_*) DLQ 滞留上限: 1000/24h

1.5.13.10 5.13.10 IF #12 運営者操作通知受信

POST /internal/admin-integration/v1/admin-operation/notify

リクエスト:

```

{
  "operationId": "...",
  "contractOwnerUserId": "...",
  "operationKind": "owner.restore_data|owner.suspend|rate_limit.override|...",
  "occurredAt": "...",
  "actorOperatorId": "...",
  "ticketId": "...",
  "summary": " 削除データを復元しました "
}

```

動作:

1. JWT + Idempotency-Key 検証
2. **inbox_messages** に重要度 **high** で生成 (管理者 inbox 表示)
3. メール通知 Queue 投入 (**critical** の場合はメール強制送信)

1.5.14 5.14 外部 Webhook(Resend 等)

1.5.14.1 5.14.1 POST /webhooks/resend 認証: Resend 署名検証 (HMAC-SHA256)

リクエスト (Resend 仕様準拠):

```
{
  "type": "email.delivered|email.bounced|email.complained|email.delivery_delayed",
  "data": {
    "message_id": "...",
    "to": "...",
    "subject": "...",
    "timestamp": "..."
  }
}
```

動作:

1. 署名検証 (失敗時は 401)
2. **notification_logs.delivery_state** を遷移
3. **bounced/complained** の場合は **email_suppression_list** に追加 (全契約横断)

エラー: 401 **SIGNATURE_INVALID**、200 + 既存処理結果 (冪等性違反)

1.6 6. 連携 IF 通信仕様一覧

IF#	連携先	方向	認証	Idempotency	DLQ 上限	TO	主管
1	顧管	→ ×	mTLS + JWT(aud=main)	(operation_id, contract_owner_user_id, target_status)	100/30 分	5s	受信
2	顧管	→ ×	mTLS + JWT	(operation_id, contract_owner_user_id)	100/30 分	3s	受信
4	顧管	→ ×	mTLS + JWT + 再 認証	(restore_op_id)	50/24h	10s	受信
5	顧管	→ ×	mTLS + JWT	(owner_id, override_id)	50/30 分	3s	受信
6	顧管	→ ×	mTLS + JWT	(owner_id, scope, version)	50/30 分	3s	受信
7	顧管	→ ×	mTLS + JWT	(announcement_id, owner_id)	1000/h	30s	受信
8	顧管	× →	mTLS + JWT(aud=admin)	(metric_window_id)	n/a	10s	送信
9	顧管	× →	mTLS + JWT(aud=admin)	(detection_id)	200/30 分	5s	送信
10	Stripe → 顧管 → ×	→ ×	Stripe 署名 + mTLS	event_id	1000/24h	10s	受信
12	顧管	→ ×	mTLS + JWT	(op_id, owner_id)	200/30 分	5s	受信

詳細は運営者側 [03_API 設計書.md §5.8](#) を参照。

1.7 7. 未決事項

No	内容	確認先	期限	ステータス
1	IF #3 / #11 の用途	要件側で予約済み、MVP 範囲外	Future	既決